

Meta-UAD: A Meta-Learning Scheme for User-level Network Traffic Anomaly Detection

Tongtong Feng¹, Qi Qi², Lingqi Guo², Jingyu Wang^{2*}

¹ Tsinghua University, Beijing, China

² SKL-NST, Beijing University of Posts and Telecommunications, Beijing, China

fengtongtong@tsinghua.edu.cn, {qiqi8266, guolingqi, wangjingyu}@bupt.edu.cn

Abstract—Accuracy anomaly detection in user-level network traffic is crucial for network security. Compared with existing models that passively detect specific anomaly classes with large labeled training samples, user-level network traffic contains sizeable new anomaly classes with few labeled samples and has an imbalance, self-similar, and data-hungry nature. Motivation on those limitations, in this paper, we propose *Meta-UAD*, a Meta-learning scheme for User-level network traffic Anomaly Detection. Meta-UAD uses the CICFlowMeter to extract 81 flow-level statistical features and remove some invalid ones using cumulative importance ranking. Meta-UAD adopts a meta-learning training structure and learns from the collection of K-way-M-shot classification tasks, which can use a pre-trained model to adapt any new class with few samples by few iteration steps. We evaluate our scheme on two public datasets. Compared with existing models, the results further demonstrate the superiority of Meta-UAD with 15% - 43% gains in F1-score.

Index Terms—Anomaly Detection, Network Traffic, Few-Shot Learning, Meta-Learning

I. INTRODUCTION

Accuracy anomaly detection in user-level traffic is crucial for network security [1]. Attackers might exploit an application containing a vulnerability, jeopardizing the confidentiality, integrity, and availability of the user's crucial information.

User-level network traffic contains sizeable new anomaly classes with few labeled samples. Those new anomaly classes possess three unique characteristics. *Imbalanced* [2], [3]: compared to the existing anomaly classes, it is expensive and arduous to collect a massive amount of data onto the new anomaly class. Therefore, the sample sizes of different anomaly classes in the sampled database are often highly imbalanced. *Self-similar* [4], [5]: the new anomaly classes evolve from the existing ones and have characteristics closer to normal traffic. As our detection models adapt, so does anomaly traffic. According to the McAfee labs threats reports¹, most new anomaly classes are branches of existing anomaly families. *Data-hungry* [6], [7]: now more attackers are focusing on fewer but more precise targets instead of widespread invasions, so each new anomaly class has small-scale samples. According to the CrowdStrike report², near 62% attackers will remain silent until a precise target is discovered, and they use

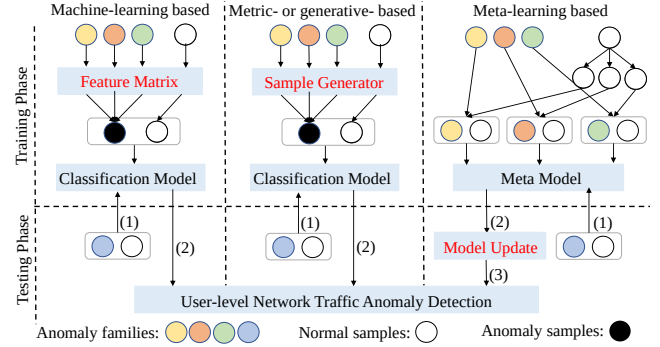


Fig. 1. Network traffic anomaly detection schemes.

legitimate credentials and built-in tools to attack, which can avoid detection by traditional detection models.

Existing anomaly detection models can be grouped into three categories (see in Fig.1). Machine-learning (ML) based models [8]–[10] classify data using a feature matrix where rows and columns correspond to each packet and feature values. They can combine the rules that differentiate anomaly traffic from normal traffic and achieve high-precision classification. Metric-based models [11]–[13] can learn a non-linear metric to generate new anomaly samples by distance to a class prototype or a class sub-space. Simultaneously, the generative-based models [14]–[16] can learn a sample generator only from seen class samples to synthesize new samples using Generative Adversarial Networks (GAN).

Existing models only detect specific anomaly classes with large labeled training samples, which are hard to detect anomaly samples in user-level network traffic. 1) They heavily rely on large-scale training samples and anomaly class distribution in training datasets [17]. They only can achieve good results for specific anomaly classes (same as the training dataset). For small-scale training datasets (e.g., endpoint traffic), the rules learned by these models may be entirely inapplicable to other datasets (e.g., cloud traffic). 2) For extremely imbalanced training datasets, models can produce an inductive learning bias towards the majority classes due to overfitting [18], resulting in poor minority class detection performance. The sample generator can alleviate the data imbalance problem. Nevertheless, the sample generator uses only seen class data and does not explicitly learn to generate

*Corresponding author.

¹<https://media.mcafeeassets.com/content/dam/npcldecommerce/en-us/docs/reports/rp-mobile-threat-report-feb-2023.pdf>.

²<https://www.crowdstrike.com/global-threat-report/>.

unseen class samples. If we detect many unseen classes, the number of synthesizing samples would be unfeasibly estimated [19], and this operation will drastically increase computational complexity and energy consumption. In the testing phase, when the new anomaly class is significantly different from anomaly classes in the training set, the performance of those models is still poor [20]. 3) Since the anomaly classes are constantly updated and training data cannot cover all anomaly classes, the effectiveness of existing models will decline sharply for new anomaly classes [21].

We argue that the recent breakthroughs in meta-learning have greatly facilitated small-scale sample processing capacities. Among them, few-shot learning [22] enables the model to effectively learn information from small samples by combining limited supervised information (few labeled samples) with prior knowledge (meta-network). Meta-SGD [23] is a dominant optimization-based few-shot learning model. Motivation on this opportunity, in this paper, we propose *Meta-UAD*, a Meta-learning scheme for User-level network traffic Anomaly Detection, which is based on Meta-SGD. Meta-UAD uses the CICFlowMeter [24] to extract 81 flow-level statistical features and remove some invalid ones using cumulative importance ranking. Meta-UAD adopts a meta-learning training structure and learns from the collection of K-way-M-shot classification tasks (see in Fig.1). To evaluate our scheme's efficiency, we conduct two comparative experiments in two public network traffic datasets, and the results further demonstrate the superiority of Meta-UAD with 15% - 43% gains in the F1-score.

II. ANOMALY DETECTION WITH META-LEARNING

A. Problem Definition

We define flow $f = \{p_1, p_2, \dots, p_u\}$ as a set of adjacent packets p sharing the same 5-tuple information: [source IP address, destination IP address, source port number, destination port number, and transport layer protocol], where u is equal to the number of the packet p in flow f . We define $x_f = \{a_f^j\}_{0 \leq j \leq v}$ as the feature vector of flow f , where a_f^j is one feature of flow f , such as the packet size, and v is the feature number of flow f . We denote by y_f the detection label of flow f . Therefore, each sample in the training dataset can be expressed as (x_f, y_f) , and each anomaly detection model can be shown: $\pi_\theta : x \rightarrow y$.

In the user-level network traffic scenarios, we have a large training dataset $\mathcal{D}^{tr}$ (typically anomaly classes with large samples) and a test dataset $\mathcal{D}^{test}$ (new anomaly classes with one or a few samples), in which their respective class sets colon $\mathcal{C}_{tr} = \{1, \dots, |\mathcal{C}_{tr}|\}$ and $\mathcal{C}_{test} = \{|\mathcal{C}_{tr}| + 1, \dots, |\mathcal{C}_{tr}| + |\mathcal{C}_{test}|\}$ are disjoint. Our problem is how to learn an anomaly detection model π_θ on $\mathcal{C}_{tr}$ that can accurately generalize to new anomaly classes $\mathcal{C}_{test}$ with few labeled samples by few computational consumptions.

B. Scheme Design

In this subsection, we describe Meta-UAD's design details, which is based on Meta-SGD [23] to identify anomaly traffic.

TABLE I
EXTRACTING 33 FLOW-LEVEL STATISTICAL FEATURES

Feature ID	Feature Name
1-2	Bwd packet count: Total, Mean
3-4	Flow packet count: Total, Mean
5	Bwd/Fwd packet total count ratio
6	Fwd header Length: Total
7	Bwd header Length: Total
8	Flow header Length: Total
9	Bwd/Fwd header total length ratio
10-13	Fwd packet Length: Total, Max, Mean, Std
14-17	Bwd packet Length: Total, Max, Mean, Std
18-21	Flow packet Length: Total, Max, Mean, Std
22	Bwd/Fwd packet total length ratio
23-26	Fwd IAT: Min, Max, Mean, Std
27-28	Bwd IAT: Max, Mean
29-31	Flow IAT: Total, Max, Mean
32	Fwd flag count: PSH
33	Flow flag count: ACK

¹ Fwd/Bwd: forward/backward flow;

² IAT: the sending time interval between two adjacent packets.

³ Flag count: the number of packets with this flag.

Feature extractor. We first use CICFlowMeter [24] to extract 81 flow-level statistical features $\tilde{A}_f^v$. Then we remove some invalid features: features' missing proportion is greater than 50%, features' entropy is equal to 0, or features' importance ranking in RF [25] is in the bottom 30% (all parameters are obtained through experimental testing). Finally, we select 33 flow-level features, as shown in TABLE I.

Task Definition. We first sample K classes from $\mathcal{C}_{tr}$, $\mathcal{C}^K \sim \mathcal{C}_{tr}$. In each episode of meta-training phase, we construct task $\mathcal{T}_i = \{\mathcal{D}_i^{sup}, \mathcal{D}_i^{val}\}$ for each class $\mathcal{C}^i$, where $\mathcal{C}^i \in \mathcal{C}^K$. For each episode, we build K tasks. $\mathcal{D}_i^{sup} = \{(x_i, y_i)_m\}$ contain M labelled flows from an anomaly class $\mathcal{C}^i$ and normal classes; $\mathcal{D}_i^{val} = \{(x_i, y_i)_n\}$ contain N labelled flows from anomaly class $\mathcal{C}^i$ and normal classes.

Meta-learning. The goal of meta-learning is to construct a pre-trained model $\pi_{(\theta, \alpha)} : x \rightarrow y$ with parameters θ, α . θ is a set of the model's initial parameters; α is a vector that decides both the model's update direction and length. The meta-learning phase includes an inner update for a special task to update the local parameters and an outer update for all tasks to update global parameters.

The inner update adjusts the local parameters θ'_i for task $\mathcal{T}_i$ through a gradient descent algorithm. For a given anomaly class $\mathcal{C}^i$ and task $\mathcal{T}_i$, we first definite a loss function on the support set $\mathcal{D}_i^{sup}$:

$$\mathcal{L}_{\mathcal{T}_i}(\pi_\theta; \mathcal{D}_i^{sup}) = \sum_{(x_i, y_i) \in \mathcal{D}_i^{sup}} L(\pi_\theta(x_i), y_i) \quad (1)$$

where $\pi_\theta(x_i)$ is a probability vector that shows the probability of sample x_i being detected as different classes; y_i is a one-hot vector that shows the actual class of the sample x_i ;

$|\pi_\theta(x_i)| = |y_i| = K + 2$, it means that there are $K + 2$ classes. Among them, there are K anomaly classes in training tasks, one normal class, and one new anomaly class in testing tasks. We define $L(\cdot)$ as the cross entropy error function:

$$L(\pi_\theta(x_i), y_i) = - \sum_{k=1}^{K+2} y_i^k \log(\pi_\theta(x_i)^k) \quad (2)$$

where $\pi_\theta(x_i)^k$ is the probability that sample x_i is predicted to the class k . Then, we use a gradient update algorithm to change the local parameters from θ to θ'_i :

$$\theta'_i = \theta - \alpha \circ \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\pi_\theta; \mathcal{D}_i^{sup}) \quad (3)$$

where the direction of $\alpha \circ \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\pi_\theta; \mathcal{D}_i^{sup})$ represents the direction of gradient update, and the length of $\alpha \circ \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}(\pi_\theta; \mathcal{D}_i^{sup})$ represents the length of gradient update.

The outer update is to update the global parameters θ, α by taking into account all the sampled tasks. For a given anomaly class $\mathcal{C}^i$ and task $\mathcal{T}_i$, we also define a loss function on the validation set $\mathcal{D}_i^{val}$:

$$\mathcal{L}_{\mathcal{T}_i}(\pi_{\theta'_i}; \mathcal{D}_i^{val}) = \sum_{(x_i, y_i) \in \mathcal{D}_i^{val}} L(\pi_{\theta'_i}(x_i), y_i) \quad (4)$$

Then, we use a gradient update algorithm to change the parameters from (θ'_i, α) to (θ', α') by taking into account all the sampled tasks:

$$(\theta', \alpha') = (\theta, \alpha) - \beta \nabla_{(\theta, \alpha)} \sum_{i=1}^K \mathcal{L}_{\mathcal{T}_i}(\pi_{\theta'_i}; \mathcal{D}_i^{val}) \quad (5)$$

where β is the learning rate of the outer update.

Meta-testing. During meta-testing, for a new anomaly class $\mathcal{C}^{new} \sim \mathcal{C}_{test}$, we first construct a support set $\mathcal{D}_{new}^{sup}$ and obtain the adapted parameters θ' with few iteration steps by the gradient update algorithm:

$$\theta' = \theta - \alpha \circ \nabla_{\theta} \mathcal{L}(\pi_{\theta}(\cdot); \mathcal{D}_{new}^{sup}) \quad (6)$$

Then, we use this updated model $\pi_{\theta'}(\cdot)$ for anomaly detection in new anomaly class $\mathcal{C}^{new}$.

Backbone architecture. Our scheme Meta-UAD is general. In theory, we can use any anomaly detection network as the backbone architecture. Because the data format of network traffic is irregular, the features of network traffic have been extracted preliminarily through the data preprocessing module. For the anomaly detection module, we only need a simple backbone architecture to learn good accuracy by using our scheme. So we come up with the Deep Neural Networks (DNN) as backbone architecture.

III. EXPERIMENTS

A. Experimental Setup

Experiments are conducted based on two public datasets: CIC-AndMal2017 [26] and CIC-IDS2017 [27]. The CIC-AndMal2017 contains 4 main malware categories and 42 malware families with 1205515 normal flows and 1411064 anomaly flows. This dataset can be viewed as imbalanced. The

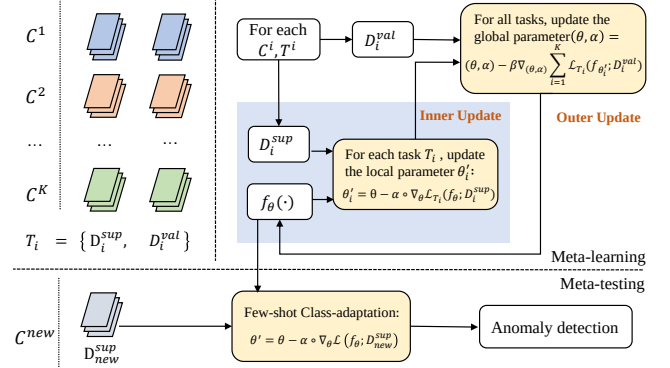


Fig. 2. User-level network traffic anomaly detection with meta-learning.

imbalanced ratios of normal flows to Pletor, FakeJobOffer, AndroidDefender, and Gooligan are 255:1, 39:1, 21:1, and 13:1, respectively. CIC-IDS2017 is provided by the Canadian Institute of Cybersecurity, which captures 2271320 normal flows and 556556 anomaly flows. Compared to CIC-AndMal2017, it is an extremely imbalanced dataset and it contains many novel attacks. The imbalanced ratios of normal flows to HeartBleed, XSS, and DoS are 206483:1, 3483:1, and 9:1, respectively.

We use DNN as the backbone architecture for our scheme, which contains three fully connected layers with hidden cells [256, 128, 128]. Outputs from these layers are then aggregated in a hidden layer that uses $K + 2$ neurons to apply the loss function. We fix the hyperparameters β in meta-learning at 0.001. During meta-training, we sample K anomaly classes from training datasets, where $K = 5$; we sample M and N flows per class, where $M = N$.

Considering the data imbalanced condition in a multiclass classification problem, to efficiently judge our scheme's performance in detecting new anomaly classes with few labeled samples, we use F1-score as the criterion. We use five ML-based models as comparison algorithms: SVM [28], DT [29], RF [25], CNN [30], LSTM [31]. Similarly, we use two metric-based models and two generative-based models as comparison algorithms: Simpleshot [12], TADAM [11], Delta-encoder [14], and Mfc-gan [15].

Suppose we have a training dataset $\mathcal{C}_{tr}$ and a test dataset $\mathcal{C}_{test}$. During the training phase, Meta-UAD samples M labeled flows per class $\mathcal{C}^i \sim \mathcal{C}_{tr}$ in each episode to construct meta-models through inner update iteration and outer update iteration. Meta-UAD then uses a pre-training meta-model to adapt to new anomaly classes $\mathcal{C}^{new}$ with M labeled samples through few iterative steps. Existing models use the entire training dataset and M labeled samples of new anomaly classes $\mathcal{C}^{new}$ for model optimization until we get the optimal model on the considered evaluation metrics. During the testing phase, we resample M labeled flows of class $\mathcal{C}^{new} \sim \mathcal{C}_{test}$ for comparing between Meta-UAD and existing models. Since the sample number in the testing set is minimal, we choose to take the mean value of multiple experiments (100 times) to achieve the reliability of the results.

Moreover, we also define an additional baseline. *Pre-*

training: the model learning from the meta-learning phase is directly applied to the meta-test phase without any adaptation. *Fine-tuning*: the model learning from the meta-learning phase is applied to the meta-test phase by a few iteration steps.

B. Meta-UAD on M-shot anomaly detection

In this subsection, to verify that Meta-UAD can accurately detect new anomaly classes with few labeled samples, we compare Meta-UAD with the existing models on the M-shot network traffic anomaly detection in terms of F1-score. In the CIC-AndMal2017 dataset, we randomly select 30 classes from 42 anomaly families as the training set C_{tr} and the remaining classes as the testing set C_{test} . We set $M = 5$, $M = 10$, $M = 20$, respectively. The experimental results are shown in TABLE II.

Our scheme is far superior to existing models in terms of F1-score. ML-based models need to be trained in large training samples, to realize high-precision classification by summarizing the rules that distinguish anomaly traffic from normal traffic. They heavily depend on the dataset's distribution and only can achieve good results for specific anomaly classes with large samples. Therefore, those models will perform extremely poorly for a new anomaly class with few samples. Since the anomaly class in the testing set has few labeled samples, those models can produce an inductive learning bias towards the majority classes. Metric-based and generative-based models can learn a sample generator using only seen class data and do not explicitly learn to generate the new class samples. If we detect many new classes, the number of synthesizing samples would be unfeasibly estimated. In the testing phase, when the new anomaly class is significantly different from anomaly classes in the training set, the performance of those models is still poor and the computational cost is very high.

We can find that the fine-tuning model is better than the pre-trained model. Before model testing, the fine-tuning model needs few iteration steps to adjust parameters and adapt to the specific new anomaly class. This operation is the core advantage of Meta-UAD.

C. Generalization

In this subsection, to verify that Meta-UAD has a strong generalization, we compared it with existing models in a cross-dataset in terms of F1-score. We select all anomaly families in CIC-AndMal2017 as the training set and anomaly families in CIC-IDS2017 as the testing set. We set $M = 20$, and the experimental results are shown in Fig. 3(a). When CIC-IDS2017 is the training set and CIC-AndMal2017 is the testing set, the experimental results are shown in Fig. 3(b).

We find that our scheme can still achieve optimal results in cross-dataset. Existing models heavily depend on the dataset's distribution and have poor generalization abilities to a new dataset. Our scheme needs to iterate some steps according to new anomaly samples in the meta-test phase, which can maximize the model's generalization and degrade the model's computational cost.

TABLE II
COMPARE META-UAD WITH EXISTING ANOMALY DETECTION MODELS ON THE M-SHOT ANOMALY DETECTION TASK.

Category	Algorithm	F1-score		
		M=5	M=10	M=20
ML-based	SVM [28]	0.554	0.602	0.695
	DT [29]	0.638	0.656	0.749
	RF [25]	0.681	0.763	0.815
	CNN [30]	0.697	0.774	0.836
	LSTM [31]	0.782	0.851	0.937
Metric-based	TADAM [11]	0.834	0.868	0.895
	Simpleshot [12]	0.848	0.879	0.934
Generation-based	Delta-encoder [14]	0.877	0.926	0.950
	Mfc-gan [15]	0.831	0.894	0.929
Ours	Pre-training	0.893	0.916	0.943
	Fine-tuning	0.949	0.966	0.981

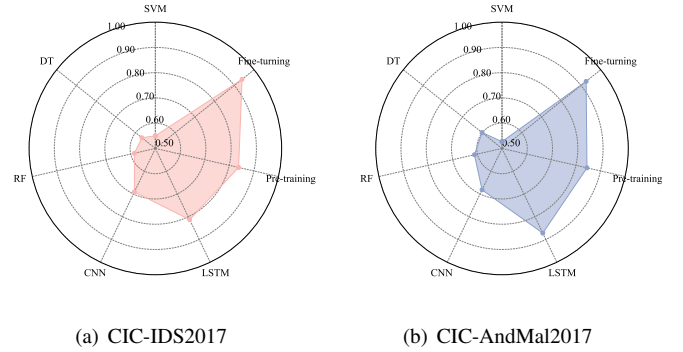


Fig. 3. Compare our scheme with existing models in cross-dataset in terms of F1-score.

IV. CONCLUSION

Anomaly detection aims to automatically identify anomaly behaviors by learning exclusively from normal traffic. Accuracy anomaly detection in user-level network traffic is crucial for network security. User-level network traffic contains sizeable new anomaly classes with few labeled samples and have an imbalance, self-similar, and data-hungry nature. For those limitations, we proposed Meta-UAD. It can use the pre-trained model to adapt new anomaly classes with few samples and going through few iteration steps. By experiment evaluation, our scheme are better than others. In the future, we hope to deploy our scheme into a real gateway.

V. ACKNOWLEDGMENTS

This work was supported in part by the National Natural Science Foundation of China under Grants (62171057, 62471055, U23B2001), China Postdoctoral Science Foundation under Grant No. 2024M751688, Postdoctoral Fellowship Program of CPSF under Grant No. GZC20240827, and Beijing Key Lab of Networked Multimedia.

REFERENCES

- [1] Y. Zhong, Z. Wang, X. Shi, J. Yang, and K. Li, "Rfg-helad: A robust fine-grained network traffic anomaly detection model based on heterogeneous ensemble learning," *IEEE Transactions on Information Forensics and Security*, 2024.
- [2] S. Gao, P. Yang, and L. Huang, "Scene-adaptive svad based on multi-modal action-based feature extraction," in *Proceedings of the Asian Conference on Computer Vision*, 2024, pp. 2471–2488.
- [3] T. Feng, Q. Qi, J. Wang, and J. Liao, "Few-shot class-adaptive anomaly detection with model-agnostic meta-learning," in *2021 IFIP Networking Conference (IFIP Networking)*, 2021, pp. 1–9.
- [4] Y. Xin, S. Luo, X. Liu, Y. Du, H. Zhou, X. Cheng, C. L. Lee, J. Du, H. Wang, M. Chen *et al.*, "V-petl bench: A unified visual parameter-efficient transfer learning benchmark," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- [5] T. Feng, Q. Qi, and J. Wang, "Multimedia traffic anomaly detection," 2024. [Online]. Available: <https://arxiv.org/abs/2408.14884>
- [6] Y. Xin, J. Du, Q. Wang, K. Yan, and S. Ding, "Mmap: Multi-modal alignment prompt for cross-domain multi-task learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 14, 2024, pp. 16 076–16 084.
- [7] T. Feng, X. Wang, F. Han, L. Zhang, and W. Zhu, "U2udata: A large-scale cooperative perception dataset for swarm uavs autonomous flight," in *Proceedings of the 32nd ACM International Conference on Multimedia*, 2024, p. 7600–7608.
- [8] T. Wang, Z. Yang, Q. Chen, Y. Sun, and C. Yan, "Rethinking pooling for multi-granularity features in aerial-view geo-localization," *IEEE Signal Processing Letters*, vol. 31, pp. 3005–3009, 2024.
- [9] Q. Chen, T. Wang, Z. Yang, H. Li, R. Lu, Y. Sun, B. Zheng, and C. Yan, "Sdpl: Shifting-dense partition learning for uav-view geo-localization," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 34, no. 11, pp. 11 810–11 824, 2024.
- [10] Y. Ge, Z. Chen, M. Yu, Q. Yue, R. You, and L. Zhu, "Mambatsr: You only need 90k parameters for traffic sign recognition," *Neurocomputing*, p. 128104, 2024.
- [11] B. Oreshkin, P. Rodriguez, and A. Lacoste, "TADAM: Task Dependent Adaptive Metric for Improved Few-shot Learning," in *NIPS*, 2018, p. 7352.
- [12] Y. Wang, W.-L. Chao, K. Q. Weinberger, and L. van der Maaten, "Simpleshot: Revisiting Nearest-neighbor Classification for Few-shot Learning," *ArXiv Preprint*, 2019.
- [13] W. Liu, S. Cheng, D. Zeng, and H. Qu, "Enhancing document-level event argument extraction with contextual clues and role relevance," *arXiv preprint arXiv:2310.05991*, 2023.
- [14] E. Schwartz, L. Karlinsky, J. Shtok, S. Harary, M. Marder, R. Feris, A. Kumar, R. Giryes, and A. M. Bronstein, "Delta-encoder: An Effective Sample Synthesis Method for Few-shot Object Recognition," *Advances in Neural Information Processing Systems*, 2018.
- [15] A. Ali-Gombe and E. Elyan, "Mfc-gan: Class-imbalanced Dataset Classification Using Multiple Fake Class Generative Adversarial Network," *Neurocomputing*, vol. 361, pp. 212–221, 2019.
- [16] W. Liu, L. Zhou, D. Zeng, Y. Xiao, S. Cheng, C. Zhang, G. Lee, M. Zhang, and W. Chen, "Beyond single-event extraction: Towards efficient document-level multi-event argument extraction," *arXiv preprint arXiv:2405.01884*, 2024.
- [17] W. Li, X. Meng, Z. Zhao, Z. Liu, C. Chen, and H. Wang, "Lot: A transformer-based approach based on channel state information for indoor localization," *IEEE Sensors Journal*, 2023.
- [18] F. Shen and J. Tang, "Imagpose: A unified conditional framework for pose-guided person generation," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [19] J. Ye, J. Zhang, and H. Shan, "Depmamba: Progressive fusion mamba for multimodal depression detection," in *ICASSP 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Hyderabad, India, April 06-11, 2025*, 2025, pp. 1–5.
- [20] F. Shen, X. Jiang, X. He, H. Ye, C. Wang, X. Du, Z. Li, and J. Tang, "Imagdressing-v1: Customizable virtual dressing," *arXiv preprint arXiv:2407.12705*, 2024.
- [21] Y. Xin, J. Du, Q. Wang, Z. Lin, and K. Yan, "Vmt-adapter: Parameter-efficient transfer learning for multi-task dense scene understanding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 14, 2024, pp. 16 085–16 093.
- [22] R. Perera and S. Halgamuge, "Discriminative sample-guided and parameter-efficient feature space adaptation for cross-domain few-shot learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 23 794–23 804.
- [23] Z. Li, F. Zhou, F. Chen, and H. Li, "Meta-SGD: Learning to Learn Quickly for Few-shot Learning," *ArXiv Preprint*, 2017.
- [24] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor Traffic Using Time based Features," in *ICISSP*, 2017, pp. 253–262.
- [25] G. Mohi-Ud-Din, L. Zhiqiang, Z. Jiangbin, W. Sifei, L. Zhijun, M. Asim, Y. Zhong, and Y. Chen, "Intrusion detection using hybrid enhanced csapo and multivariate wls random-forest technique," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 4937–4950, 2023.
- [26] A. H. Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani, "Toward Developing a Systematic Approach to Generate Benchmark Android Malware Datasets and Classification," in *ICCST*, 2018, pp. 1–7.
- [27] I. Sharafaldin, A. H. Lashkari, and A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *ICISSP*, 2018.
- [28] A. Ghorbani and S. M. Fakhrahmad, "A deep learning approach to network intrusion detection using a proposed supervised sparse auto-encoder and svm," *Iranian Journal of Science and Technology, Transactions of Electrical Engineering*, vol. 46, no. 3, pp. 829–846, 2022.
- [29] K. H. Kim, B. I. Kwak, M. L. Han, and H. K. Kim, "Intrusion detection and identification using tree-based machine learning algorithms on dcs network in the oil refinery," *IEEE Transactions on Power Systems*, vol. 37, no. 6, pp. 4673–4682, 2022.
- [30] K. L. Narayanan and R. Naresh, "Detection and prevention of black hole attack using tree hierarchical deep convolutional neural network and enhanced identity-based encryption in vehicular ad hoc network," *IEIE Transactions on Smart Processing & Computing*, vol. 13, no. 1, pp. 41–50, 2024.
- [31] Z. Chen, Y. Lou, P. He, P. Xu, and X. Zhang, "Magnetic anomaly detection based on attention-bi-lstm network," *IEEE Transactions on Instrumentation and Measurement*, 2024.